Autonomous QA — Backend × Provider-Mix Matrix

A fully autonomous QA harness that drives the Lava Android client through a real onboarding → search → open-details → obtain-download flow against a real backend and the real trackers, recording everything and vision-analyzing the recordings for the non-crashing defect classes a green JUnit verdict can hide.

- **Plan / spec:** `docs/superpowers/plans/2026-06-29-autonomous-qa-backend-provider-matrix.md`
- **on-device backend notes:** `docs/autonomous-qa/apiapp-backend-notes.md`
- **scripts:** `scripts/autonomous-qa/`

1. Purpose

Run one matrix cell at a time:

```
backend ∈ {goapi, apiapp}   (ONE at a time — never both APIs concurrently)
  × provider-subset          (all 31 non-empty subsets of the 5 tracked providers)
  × query ∈ {1080p, mp3}
```

`{RuTracker, RuTor, IPTorrents, NNMClub, Kinozal}` → 2^5 − 1 = 31 subsets. Full matrix = 31 × 2 backends × 2 queries = 124 functional iterations on a single containerized-KVM Android emulator (AVD `default`, API 34, x86_64).

Every iteration does a **fresh client install** (clean onboarding state), records **video + logcat + the gradle connected-test log**, runs the parameterized `Challenge70AutonomousQaProviderMatrixTest`, and is later **vision-analyzed**. Anti-bluff is load-bearing (§6.J / §6.L / §6.AK): a PASS must rest on real user-visible state captured on a real emulator against a real backend + real tracker; cold-start-only evidence never green-lights a distribute.

2. Orchestrator scripts (`scripts/autonomous-qa/`)

Script	Role
`run-matrix.sh`	Top orchestrator. One backend at a time.
`run-iteration.sh`	One iteration against an already-booted emulator + up backend.
`lib-subsets.sh`	Emits the 31 provider subsets.
`lib-emulator.sh`	Boots the §6.X containerized-KVM emulator, wires host adb.
`lib-backend.sh`	Brings exactly ONE backend up/down.
`vision-analyze.sh`	ffmpeg/OCR/logcat defect analysis of recordings.
`aggregate-evidence.sh`	Rolls verdicts into the cycle summary + §6.AK gate artifacts.

`lib-subsets.sh` — `qa_emit_subsets()` walks a bitmask over `QA_PROVIDERS=(rutracker rutor iptorrents nnmclub kinozal)` and prints `<csv>|<slug>|<statehash>` per non-empty subset (31 lines). `statehash` is the short sha1 of the csv (the §11.4.128 state-hash for the evidence layout).

`lib-emulator.sh` — thin Lava glue replicating the Containers submodule's `pkg/emulator/containerized.go`. The emulator process runs **INSIDE** a podman container (`EMU_IMAGE`, default `ghcr.io/vasic-digital/lava-android-emulator`), never host-direct, never a live device. Functions: `emu_pick_port`, `emu_boot` (`podman run -d --device /dev/kvm`, maps a host port to the container's adb bridge, persists `.emu-state`), `emu_authorize_adb` (copies the image's baked adbkey out, points the host adb client at it), `emu_connect`, `emu_wait_boot` (polls `sys.boot_completed` + `pm path android`), `emu_cleanup_orphans` (removes leftover `lava-emu-*` containers holding a KVM slot), `emu_teardown`.

`lib-backend.sh` — a `.backend-active` lock enforces mutual exclusion (refuses to bring a backend up while another is active). `backend_up_goapi` brings the Go API up via `./start.sh` (the lava-containers Go orchestrator — raw podman-compose cannot handle the `network_mode: host` services) and polls `https://127.0.0.1:8443/health`. `backend_target_goapi` runs `adb reverse` and prints the client URL. `backend_up_apiapp` installs + launches the on-device `:api-app`. (`*_down_*` tear each down + clear the lock. See `apiapp-backend-notes.md` for the corrected, deterministic api-app bring-up — the in-tree `backend_up_apiapp` still carries a marked `sleep 25` readiness gap.)

`run-iteration.sh` — for one `{backend, providers, query, serial, api-url, evidence-dir}`: (1) uninstall+install the client APK; (2) clear logcat, start a threadtime logcat capture + a chunked `screenrecord` loop into `raw/`; (3) run `:app:connectedDebugAndroidTest` with the Challenge class + `qa_*` instrumentation args; (4) stop recording (EXIT-trapped); (5) parse the JUnit XML into a curated `verdict.json` (PASS/FAIL/SKIP). Exits 0 only on PASS.

`run-matrix.sh` — backend up → boot ONE emulator (kept alive for the whole matrix) → authorize adb + wait boot → backend targeting → loop every (subset × query) through `run-iteration.sh` → tear emulator + backend down (EXIT-trapped) → write the cycle `summary.md` table with per-cell verdicts and PASS/SKIP/FAIL totals.

`vision-analyze.sh` — reads each iteration's `raw/` and emits a curated `vision-analysis.md` flagging §6.AB non-crashing defects: blank/white/monochrome render (ffmpeg `signalstats`), stuck/frozen screen (ffmpeg `freezedetect`), wrong-screen / missing-download-affordance (tesseract OCR, only if present), and crashes/ANRs (logcat grep). `--iteration-dir` analyzes one cell; `--cycle-dir` rolls up a backend. Anti-bluff: ffmpeg is required (else exit 2, no verdict); unreadable recordings or missing logcat yield `INCOMPLETE`, never a faked `CLEAN`; OCR is honestly skipped when tesseract is absent. Exit: 0 CLEAN, 1 DEFECTS-FOUND, 2 CANNOT-ANALYZE/INCOMPLETE.

`aggregate-evidence.sh` — rolls every `verdict.json` into a cycle `CYCLE-SUMMARY.md` (per-backend + overall totals) plus the §6.AK distribute-gate artifacts `check-cycle-coverage.sh` consumes: `cycle-coverage-map-<VER>.yaml` (only PASS iterations become asserted claims) and `<VER>-test-evidence.json` (every iteration as a `test_results` row). Numbers are derived from the on-disk verdicts; a missing/corrupt verdict.json is counted FAIL and noted, never silently skipped.

3. How to run

Prereqs: podman 5.x + `/dev/kvm`, the emulator container image, the Android SDK platform-tools, ffmpeg (tesseract optional for OCR), tracker creds in `.env`, and built APKs (`./gradlew :app:assembleDebug`, and `:api-app:assembleDebug` for the apiapp backend).

```
# Keystone (Phase 0) — one Go-backend cell, RuTor / 1080p:
scripts/autonomous-qa/run-matrix.sh --backend goapi --subsets
        rutor --queries 1080p

# Full Go-API matrix (31 subsets × 2 queries):
scripts/autonomous-qa/run-matrix.sh --backend goapi --subsets all
        --queries 1080p,mp3
```

```
# On-device :api-app backend matrix:
scripts/autonomous-qa/run-matrix.sh --backend apiapp --subsets
        all --queries 1080p,mp3
```

`--subsets` accepts `all` or `;`-separated csv groups, e.g. `"rutracker;rutor,kinozal"`. Defaults: `--backend goapi`, `--queries 1080p,mp3`, `--subsets all`.

Analyze + aggregate after a run:

```
scripts/autonomous-qa/vision-analyze.sh --cycle-dir .lava-ci-
        evidence/autonomous-qa/<date>/goapi
scripts/autonomous-qa/aggregate-evidence.sh --date <date> --
        version <ver> \
  --channel debug --timestamp <ISO8601>
```

4. Evidence layout

```
.lava-ci-evidence/autonomous-qa/<date>/
├─ <backend>/                                # goapi | apiapp
│  ├─ <slug>-<query>/                        # e.g. rutor-
kinozal-1080p
│  │  ├─ raw/                                # GITIGNORED
│  │  │  ├─ rec_*.mp4 / qa_rec_*.mp4         #   chunked
screenrecord
│  │  │  ├─ logcat.txt                       #   threadtime logcat
│  │  │  └─ gradle-connected.log             #
connectedDebugAndroidTest stdout
│  │  ├─ verdict.json                        # TRACKED (curated
JUnit verdict)
│  │  ├─ junit.xml                           # TRACKED (small)
│  │  └─ vision-analysis.md                  # TRACKED (curated)
│  └─ summary.md                             # TRACKED (cycle table
+ totals)
└─ CYCLE-SUMMARY.md                          # TRACKED (both
backends, by aggregate)
```

`<date>/<backend>/<slug>-<query>/` is the §11.4.128 deterministic recording layout. `aggregate-evidence.sh` additionally writes the §6.AK gate files (`cycle-coverage-map-<ver>.yaml`, `<ver>-test-evidence.json`) under `.lava-ci-evidence/distribute-changelog/firebase-app-distribution/`.

5. Constitutional context

- **§6.AH / §6.AG / §6.X — emulator in a container only.** The emulator process runs inside a podman container managed via the Containers submodule's containerized-KVM model (`--device /dev/kvm`). Never host-direct, never a live ADB device (physical devices are assumed in use by other projects).

- **§6.AK — cycle-coverage gate.** Only PASS iterations become asserted claims in the coverage map; the gate refuses a distribute whose CHANGELOG claims a fix with no executed+passed covering Challenge on the same SHA.
- **§6.H — credentials in `.env` only.** The 5 tracker logins live only in the gitignored `.env`; they are **never** echoed to logs, screenshots, commits, or `${VAR:-...}` shell expansions, and never appear in this harness's output.
- **Gitignore policy.** Raw video + live logs (`autonomous-qa/**/raw/`, `**/api-logs/`, `**/*.mp4`, `**/*.webm`) are git-excluded by design (operator directive 2026-06-29). Only curated `verdict.json` / `junit.xml` / `vision-analysis.md` / `summary.md` / `CYCLE-SUMMARY.md` are tracked.
- **Local-Only CI/CD.** The harness is bash + adb + podman + gradle on the operator's host; no hosted CI.

6. Backend reachability

- **Go API (`goapi`):** brought up on the host at `https://127.0.0.1:8443`. `backend_target_goapi` runs `adb -s <serial> reverse tcp:8443 tcp:8443`, so the emulator's `127.0.0.1:8443` tunnels over the adb transport to the host Go API (topology-independent for a containerized emulator). The client onboards against `https://127.0.0.1:8443`.
- **On-device API (`apiapp`):** `:api-app` is installed + launched **on the emulator**; the client reaches the embedded server over on-device loopback (`127.0.0.1:8443`) — no `adb reverse` needed. Readiness is a real served-surface `/health` probe via an `adb forward` host tunnel (see `apiapp-backend-notes.md`).